

FPGA Time Tagger

Complete Working Principle & Code Guide

Target: Xilinx UltraScale+ (Zynq MPSoC)

Tools: PetaLinux + Vivado 2022.2+

Repository: github.com/chandrika2708/time_tagger_kiro

Multi-channel Time-to-Digital Converter with ~10 ps resolution

8 channels | CARRY8 delay line | AXI DMA | Python interface

Table of Contents

1. System Working Principle
2. RTL Module Code Descriptions
3. Verification Suite
4. FPGA Board Testing Guide (PetaLinux)
5. Python Host Interface
6. Quick Reference

1. System Working Principle

1.1 What is a Time Tagger?

A Time Tagger is a precision instrument that records the exact time when electrical events (pulses) arrive at its input channels. It is used in quantum optics (photon correlation, HBT experiments), LIDAR (time-of-flight), fluorescence lifetime imaging (FLIM), and nuclear/particle physics.

1.2 How This Design Works

The system timestamps events with ~10 picosecond resolution using a tapped delay line technique implemented with Xilinx CARRY8 primitives.

Step-by-step operation:

- 1. Event arrives at an input channel (electrical pulse on FPGA pin)
- 2. CARRY8 delay chain propagates the event through 256 taps (~8 ps each)
- 3. Clock edge samples all 256 taps simultaneously -> thermometer code
- 4. Bubble correction fixes single-bit glitches in the thermometer code
- 5. Priority encoder converts thermometer code to binary (0-255) = fine timestamp
- 6. Coarse counter (48-bit at 500 MHz) provides the integer clock count
- 7. Tag formatter assembles 96-bit Tag_Record
- 8. Calibration LUT corrects for non-uniform tap delays (DNL correction)
- 9. Tag FIFO buffers the record (async: 500 MHz write -> 250 MHz read)
- 10. Tag Mux arbitrates 9 sources (8 channels + coincidence) round-robin
- 11. DMA Engine bursts tags to DDR memory via AXI4
- 12. Linux reads DDR and Python parses the binary tag records

1.3 Timing Resolution

- Coarse resolution: 2 ns (500 MHz clock period)
- Fine resolution: $2 \text{ ns} / 256 \text{ taps} = 7.8 \text{ ps per tap}$
- After calibration: ~10 ps RMS jitter

1.4 Key Specifications

Parameter	Value
Number of channels	8
Timing resolution	~10 ps
Coarse counter	48 bits
Fine taps	256 (CARRY8)
Tag record width	96 bits
FIFO depth/channel	16,384 entries
Max tag rate	80 Mtags/s
DMA burst size	Up to 256 beats
AXI data width	128b (DMA), 32b (reg)
Clock domains	500/250/100 MHz

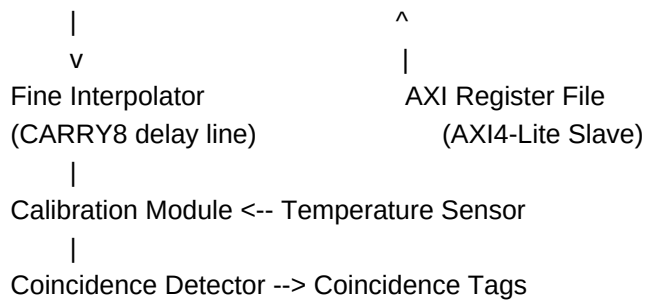
1.5 Tag_Record Format (96 bits)

Each timestamp event produces a 96-bit record:

Bits	Field	Width	Description
[95:48]	Coarse	48b	500 MHz counter value
[47:32]	Fine	16b	Calibrated fine timestamp
[31:24]	Channel ID	8b	0-7 or 0xFF (coincidence)
[23:16]	Flags	8b	Bit7:overflow, Bit0:polarity
[15:0]	Reserved	16b	Zero or coinc bitmask

1.6 System Block Diagram

Event Inputs -> TDC Channels (x8) -> Tag FIFOs -> Tag Mux -> DMA Engine -> AXI4 Master



2. RTL Module Code Descriptions

2.1 fine_interpolator.v - Delay Line TDC Core

Measures the fractional clock period between an event edge and the next clock edge using a CARRY8 tapped delay line (256 taps).

1. CARRY8 chain: 32 primitives x 8 taps = 256 total taps
2. Sampling registers capture tap outputs on clock edge
3. Bubble correction fixes isolated 0s in thermometer code
4. Priority encoder: fine_bin = index of highest set bit
5. Pipeline latency: 3 clock cycles

2.2 tdc_channel.v - Complete TDC Channel

Combines edge detection, coarse counting, fine interpolation, dead time enforcement (4 ns), calibration LUT, and tag formatting.

Edge detector -> Dead time check -> Coarse+Fine assembly
-> Calibration LUT lookup -> Tag_Record output
Supports rising+falling edges, sync_reset, channel enable

2.3 tag_fifo.v - Asynchronous FIFO

Bridges 500 MHz write domain to 250 MHz read domain with 16,384-entry depth and gray-code CDC pointers.

Dual-port RAM + Gray-code write/read pointers
High-watermark at 75% (12,288 entries)
Circular buffer on overflow (oldest discarded)
Occupancy accurate to +/-1 entry

2.4 tag_mux.v - Round-Robin Arbiter

Fair arbiter combining 9 tag sources (8 channels + coincidence) into a single output stream with backpressure support.

Scans from last_granted+1, grants first non-empty source
Holds output stable when tag_ready deasserted
Guarantees no data loss and per-source ordering

2.5 coincidence_detector.v - Window Comparator

Detects temporal coincidences between events on different channels within configurable time windows. 4 groups, priority-based output.

Compares timestamps within same group
If $|ts_A - ts_B| < window$ -> coincidence tag (ch_id=0xFF)
Group 0 highest priority, channel bitmask in reserved field

2.6 calibration_module.v - Auto-Calibration

Automatically calibrates fine interpolator to correct delay line non-linearity. Temperature-triggered recalibration ($dT > 5C$).

FSM: IDLE -> ACCUMULATE -> COMPUTE -> CHECK_DNL -> UPDATE_LUT
Histogram: 2.56M samples across 256 bins
Atomic LUT update (all channels simultaneously)

Old LUT active during recalibration (no gaps)

2.7 axi_register_file.v - Configuration Interface

AXI4-Lite slave with read-write and read-only registers. SLVERR on writes to RO registers. Special STATUS/FIFO_DATA behavior.

```
RW: CTRL, CH_ENABLE, EDGE_CONFIG, DMA_CTRL, COINC_*
RO: CLK_STATUS, CAL_STATUS, DMA_STATUS (SLVERR on write)
STATUS write -> err_clear_strobe pulse
FIFO_DATA read -> fifo_rd_en pulse
```

2.8 axi_dma_engine.v - Burst DMA

Transfers tags to DDR via AXI4 burst writes. Supports circular buffer addressing and partial bursts on timeout.

```
FSM: IDLE -> COLLECT -> ADDR -> DATA -> RESP
Each 96-bit tag zero-padded to 128 bits (one AXI beat)
Circular wrap at base_addr + buf_size
awsize=4 (16 bytes), awburst=INCR
```

2.9 rate_monitor.v - Health Monitoring

Tracks per-channel tag rates (1 ms window), detects overflow, maintains error counters with saturation.

```
Per-channel 16-bit rate counter (saturates at 0xFFFF)
Aggregate overflow: >80 Mtags/s sustained >1 us
Error counter: 32-bit, saturates at 0xFFFFFFFF
Status: 00=OK, 01=disabled, 10=overflow, 11=error
```

2.10 clock_manager.v - Clock Generation

Generates all internal clocks from board oscillator using MMCME4_ADV. Supports external 10 MHz reference with automatic failover.

```
State machine: IDLE->INTERNAL->EXT_LOCKING->EXT_LOCKED->FALLBACK
Outputs: 500 MHz, 250 MHz, 100 MHz
BUFGMUX_CTRL for glitch-free clock switching
clk_loss_error asserted on external reference loss
```

3. Verification Suite

The project includes 11 self-checking testbenches in `tb/` that run with Icarus Verilog 12.0 (`iverilog -g2012`). Each testbench reports [PASS]/[FAIL] per check and exits with code 0 (all pass) or 1 (failures).

3.1 Testbench List

TB#	File	Verification Goals
1	<code>tb_fine_interpolator.v</code>	Delay line, bubble correction,
2	<code>tb_tdc_channel.v</code>	Edge detect, dead time, tag fo
3	<code>tb_tag_fifo.v</code>	Async FIFO, CDC, overflow
4	<code>tb_tag_mux.v</code>	Round-robin, backpressure, no
5	<code>tb_coincidence_detector.v</code>	Window detection, priority
6	<code>tb_calibration_module.v</code>	Histogram, DNL, LUT, temperatu
7	<code>tb_axi_register_file.v</code>	AXI4-Lite protocol, SLVERR
8	<code>tb_axi_dma_engine.v</code>	Burst writes, circular buffer
9	<code>tb_rate_monitor.v</code>	Rate counting, saturation
10	<code>tb_clock_manager.v</code>	Clock gen, failover, FSM
11	<code>tb_time_tagger_top.v</code>	End-to-end data flow

3.2 Running Testbenches

Run in WSL (Ubuntu) with Icarus Verilog installed:

```
# Example: Fine Interpolator
iverilog -g2012 -I rtl -o sim/tb_fine_interpolator.vvp \
  sim/xilinx_stubs.v rtl/time_tagger_pkg.v \
  rtl/fine_interpolator.v tb/tb_fine_interpolator.v
vvp sim/tb_fine_interpolator.vvp

# Run all (bash script):
for tb in fine_interpolator tdc_channel tag_fifo ...; do
  iverilog -g2012 -I rtl -o sim/tb_${tb}.vvp \
    sim/xilinx_stubs.v rtl/time_tagger_pkg.v rtl/*.v \
    tb/tb_${tb}.v && vvp sim/tb_${tb}.vvp
done
```

4. FPGA Board Testing Guide (PetaLinux)

4.1 Prerequisites

- Xilinx Zynq UltraScale+ board (ZCU102, ZCU104, KCU116)
- Vivado 2022.2+ (for bitstream generation)
- PetaLinux 2022.2+ (for Linux image)
- SD card (16 GB+)
- Ethernet cable or USB cable for data transfer
- Signal source (pulse generator or photon detector)

4.2 Step 1: Vivado Project Setup

Create a Vivado project, add all RTL from rtl/, set time_tagger_top as top module, and add constraints/time_tagger.xdc.

Block Design connections:

- PS M_AXI_HPM0_FPD -> s_axi_* (register access at 0xA000_0000)
- m_axi_* -> PS S_AXI_HP0_FPD (DMA writes to DDR)
- event_in[7:0] -> external FPGA pins (from XDC)
- clk_board -> 100 MHz oscillator on board

4.3 Step 2: PetaLinux Build

```
# Create project
petalinux-create -t project --template zynqMP -n time_tagger_linux
cd time_tagger_linux

# Import hardware
petalinux-config --get-hw-description=/path/to/time_tagger_top.xsa

# Enable UIO driver in kernel config
petalinux-config -c kernel
# -> Device Drivers -> Userspace I/O -> Enable

# Enable Python in rootfs
petalinux-config -c rootfs
# -> Filesystem Packages -> misc -> python3

# Build
petalinux-build

# Package boot image
petalinux-package --boot --fsbl zynqmp_fsbl.elf \
  --fpga system.bit --pmufw pmufw.elf --u-boot
```

4.4 Step 3: Device Tree Configuration

Add to system-user.dtsi for UIO and DMA buffer:

```
/include/ "system-conf.dtsi"
/ {
    reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;
```

```

ranges;
dma_buf: buffer@10000000 {
    compatible = "shared-dma-pool";
    reg = <0x0 0x10000000 0x0 0x1000000>;
    no-map;
};
};
};
&amba {
    time_tagger@a0000000 {
        compatible = "generic-uio";
        reg = <0x0 0xa0000000 0x0 0x10000>;
    };
};
};

```

4.5 Step 4: Flash and Boot

```

# Partition SD card:
#   Part 1: FAT32, 512 MB (boot)
#   Part 2: ext4, remaining (rootfs)

# Copy boot files to FAT32 partition:
cp BOOT.BIN image.ub boot.scr /media/BOOT/

# Extract rootfs to ext4 partition:
sudo tar xf rootfs.tar.gz -C /media/rootfs/

# Copy Python script:
cp python/time_tagger_host.py /media/rootfs/home/root/

# Set board DIP switches to SD boot mode
# Power on -> Linux boots -> login as root

```

4.6 Step 5: Verify on Board

```

# Check UIO device appeared
ls /dev/uio0

# Check address mapping
cat /sys/class/uio/uio0/maps/map0/addr # 0xa0000000
cat /sys/class/uio/uio0/maps/map0/size # 0x10000

# Run time tagger (local mode - prints to console)
python3 time_tagger_host.py --mode local --uio --duration 5

# Run as TCP server (for remote access)
python3 time_tagger_host.py --mode ethernet --port 5555 --uio

```

5. Python Host Interface

5.1 Architecture

The Python script (`python/time_tagger_host.py`) runs on the Zynq PS ARM core under PetaLinux. It accesses FPGA registers via memory-mapped I/O (`/dev/mem` or `/dev/uio0`) and reads DMA buffers from DDR. It can serve data over TCP (Ethernet) or USB serial to a remote host PC.

Data Flow:

```
FPGA Event -> TDC -> DMA -> DDR Memory
-> Python reads DDR (mmap)
-> Parses 96-bit Tag_Records
-> Sends over TCP/USB to Host PC
```

5.2 Running on FPGA Board (Server)

```
# Ethernet mode (TCP server on port 5555)
python3 time_tagger_host.py --mode ethernet --port 5555 --uio

# USB serial mode
python3 time_tagger_host.py --mode usb --device /dev/ttyGS0 --uio

# Local mode (print to console)
python3 time_tagger_host.py --mode local --uio --duration 10

# Options:
# --channels 0xFF      Enable all 8 channels
# --channels 0x03      Enable only channels 0,1
# --uio                Use /dev/uio0 (no root needed)
# --duration 30        Run for 30 seconds
```

5.3 Running on Host PC (Client)

```
# Receive via Ethernet
python3 time_tagger_host.py --mode client \
  --host 192.168.1.100 --port 5555 --duration 10

# Receive via USB serial
python3 time_tagger_host.py --mode client \
  --device /dev/ttyACM0 --duration 10
# (Windows: --device COM3)

# Save raw data to file
python3 time_tagger_host.py --mode client \
  --host 192.168.1.100 --output timestamps.bin
```

5.4 Data Analysis Example

```
import struct, numpy as np

# Read saved binary data (10 bytes per tag)
with open("timestamps.bin", "rb") as f:
    raw = f.read()

n_tags = len(raw) // 10
timestamps = np.zeros(n_tags, dtype=np.uint64)
channels = np.zeros(n_tags, dtype=np.uint8)
```

```

for i in range(n_tags):
    ts, ch, flags = struct.unpack_from("<QBB", raw, i*10)
    timestamps[i] = ts # picoseconds
    channels[i] = ch

# Time differences on channel 0
ch0 = timestamps[channels == 0]
dt_ns = np.diff(ch0) / 1000.0 # convert ps to ns
print(f"Mean: {dt_ns.mean():.2f} ns, Std: {dt_ns.std():.2f} ns")

```

5.5 USB Gadget Setup

To use USB serial mode, configure USB gadget on the FPGA board:

```

modprobe libcomposite
modprobe usb_f_acm
mkdir -p /sys/kernel/config/usb_gadget/tagger
cd /sys/kernel/config/usb_gadget/tagger
echo 0x1d6b > idVendor
echo 0x0104 > idProduct
mkdir -p strings/0x409
echo "TimeTagger" > strings/0x409/product
mkdir -p configs/c.1
mkdir -p functions/acm.usb0
ln -s functions/acm.usb0 configs/c.1/
echo "fe200000.dwc3" > UDC
# Now /dev/ttyGS0 is available

```

6. Quick Reference

6.1 Register Map

Offset	Name	R/W	Purpose
0x000	CTRL	RW	Bit0: enable, Bit1: sync
0x004	STATUS	RW	Write clears errors
0x008	CH_ENABLE	RW	Channel enable mask [7:0]
0x00C	EDGE_CONFIG	RW	Falling edge enable [7:0]
0x010	CLK_STATUS	RO	Clock lock/loss status
0x014	CAL_CTRL	RW	Calibration control
0x020	COINC_GROUP0	RW	Bit31:en, [7:0]:mask
0x040	COINC_WIN0	RW	Window value [9:0]
0x100	FIFO_DATA	RO	Read triggers fifo_rd_en
0x200	DMA_STATUS	RO	DMA busy/error
0x204	DMA_CTRL	RW	Bit0:en, [15:8]:burst-1
0x300	DEAD_TIME	RW	Dead time config

6.2 Directory Structure

```

time_tagger_kiro/
  rtl/           RTL source files (12 modules)
  tb/           Self-checking testbenches (11)
  sim/          Xilinx primitive stubs
  constraints/   FPGA pin constraints (XDC)
  docs/         Documentation
  python/       Host-side Python interface

```

6.3 Network Diagram

```

+-----+ Ethernet +-----+
| Host PC | <-----> | FPGA Board |
|         | TCP:5555 | (Zynq MPSoC) |
| Python  |         | Python server |
| client mode | OR USB | time_tagger_ |
|         | <-----> | host.py    |
|         | /dev/ttyACM0 |         |
+-----+           +-----+

```

6.4 Connecting Event Sources

Connect your signal source to the FPGA pins mapped to event_in[7:0] in the XDC constraints file. Requirements:

- Logic levels: LVCMOS33 or LVTTTL (3.3V)
- Pulse width: minimum 2 ns (one clock cycle at 500 MHz)
- Proper termination to avoid reflections
- Single-photon detectors: TTL output directly compatible
- Pulse generators: use 50 ohm termination